



— Montreal, June 15–19 —

DLM|2026

Hands-On: Vision Foundation Models for Computational Pathology

Omprakash Chakraborty / Valentin Gautier / Jose Dolz



Fonds de recherche
Nature et
technologies

Québec



CREATIS



Objectives of the Hands-On

- ❖ Understand pathology image representations
- ❖ Learn how foundation models are trained
- ❖ Extract embeddings using PLIP
- ❖ Perform zero-shot classification
- ❖ Compare zero-shot and supervised learning
- ❖ Train a simple classifier on extracted features

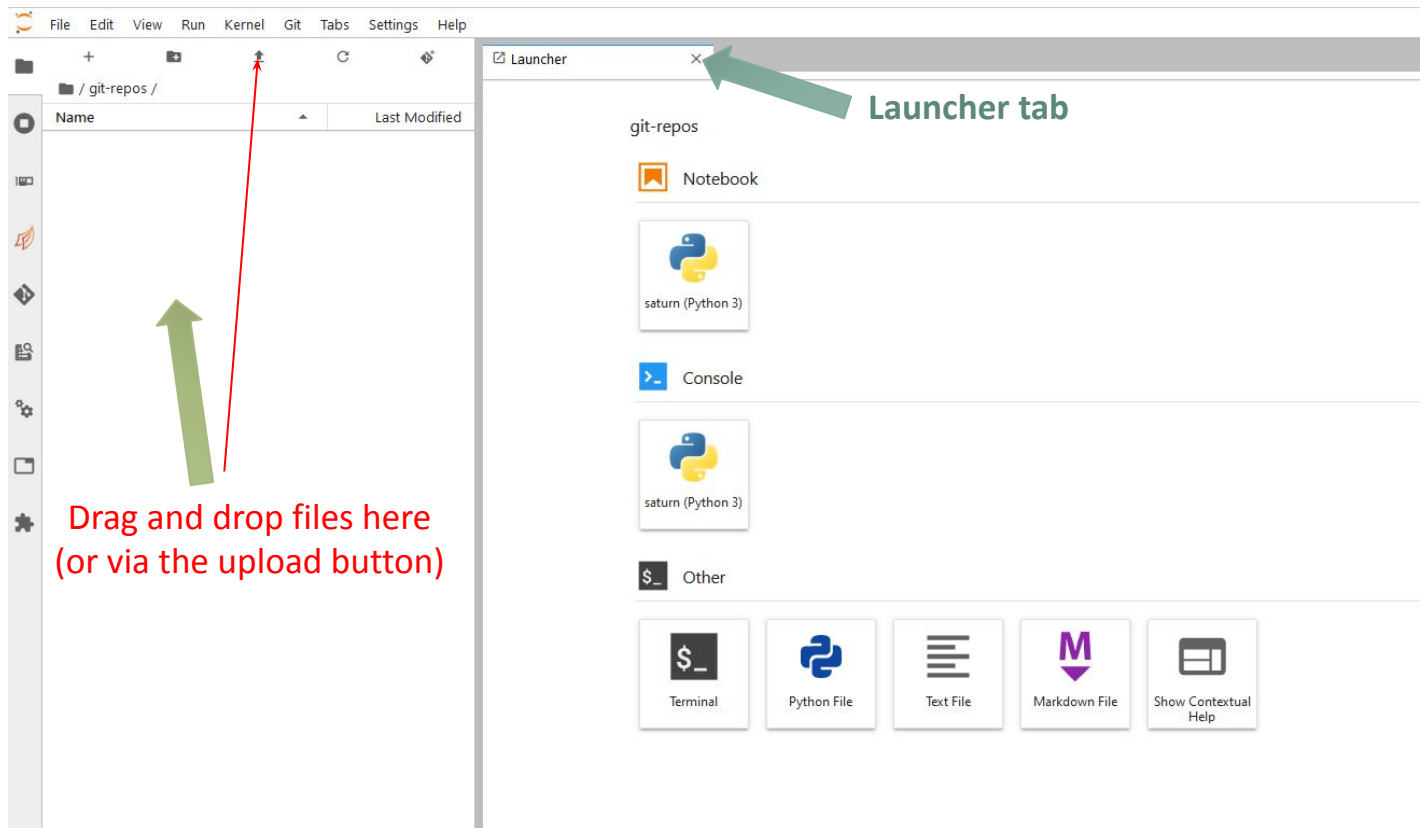
Steps required

1. Have an account on CalculQuebec Cloud and log-in
<https://jupyter.dmlti.calculquebec.cloud/hub/login>
2. Create a compute node with
 1. Memory at 4096MB
 2. GPU 1x1G.5GB

Steps covered here

3. Start the jupyter-lab server and wait for it
4. Download the hands-on material into your jupyter-lab
5. Open a notebook
6. Cells philosophy and execution

3- Jupyter Lab interface (can change)



4- Download Hands-on materials : 2 ways

Way « on server »

a- Open a **Terminal**

b- download from the terminal with :

```
git clone
https://github.com/omchakrabart
y/DIML_SummerSchool.git
```

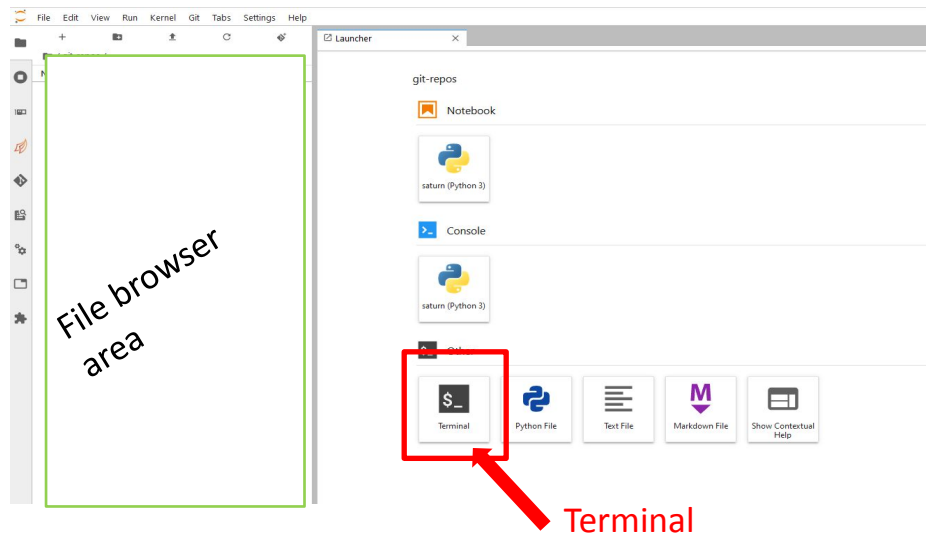
5

Way « Drag and drop » (recommended)

a- download on your computer the hands-on material as zip file (links? : next page!)

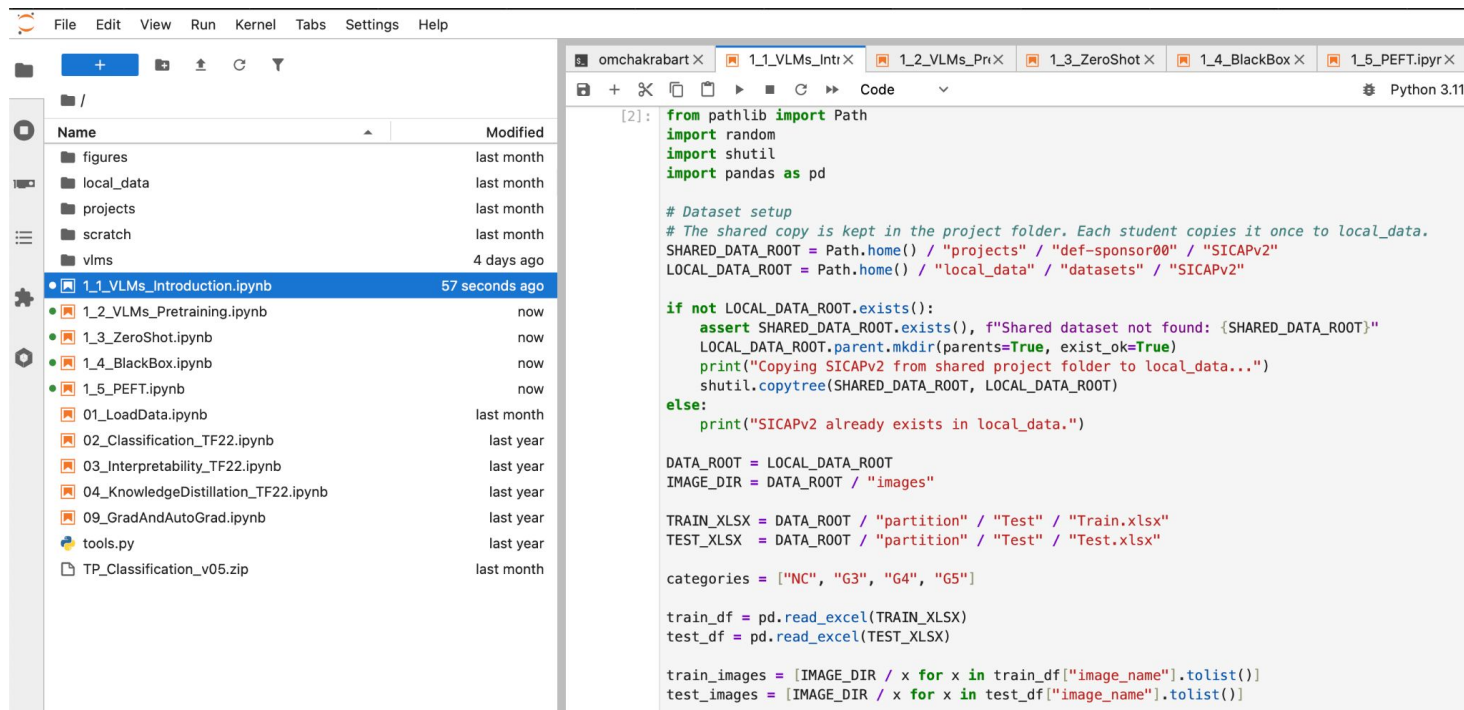
b- drag on drop it in the **file browser area**

c- unzip from jupyter lab (in a Terminal)



5- Select and open the desired notebook

As an example, double click on : 1_1_VLMs_Introduction.ipynb



The screenshot displays the JupyterLab interface. On the left, the file explorer shows a directory structure with folders like 'figures', 'local_data', 'projects', 'scratch', and 'vlms'. A list of notebooks is shown below, with '1_1_VLMs_Introduction.ipynb' highlighted. On the right, the code editor shows the content of the selected notebook, which includes imports for Path, random, shutil, and pandas, followed by dataset setup logic and data loading code.

```
[2]: from pathlib import Path
import random
import shutil
import pandas as pd

# Dataset setup
# The shared copy is kept in the project folder. Each student copies it once to local_data.
SHARED_DATA_ROOT = Path.home() / "projects" / "def-sponsor00" / "SICAPv2"
LOCAL_DATA_ROOT = Path.home() / "local_data" / "datasets" / "SICAPv2"

if not LOCAL_DATA_ROOT.exists():
    assert SHARED_DATA_ROOT.exists(), f"Shared dataset not found: {SHARED_DATA_ROOT}"
    LOCAL_DATA_ROOT.parent.mkdir(parents=True, exist_ok=True)
    print("Copying SICAPv2 from shared project folder to local_data...")
    shutil.copytree(SHARED_DATA_ROOT, LOCAL_DATA_ROOT)
else:
    print("SICAPv2 already exists in local_data.")

DATA_ROOT = LOCAL_DATA_ROOT
IMAGE_DIR = DATA_ROOT / "images"

TRAIN_XLSX = DATA_ROOT / "partition" / "Test" / "Train.xlsx"
TEST_XLSX = DATA_ROOT / "partition" / "Test" / "Test.xlsx"

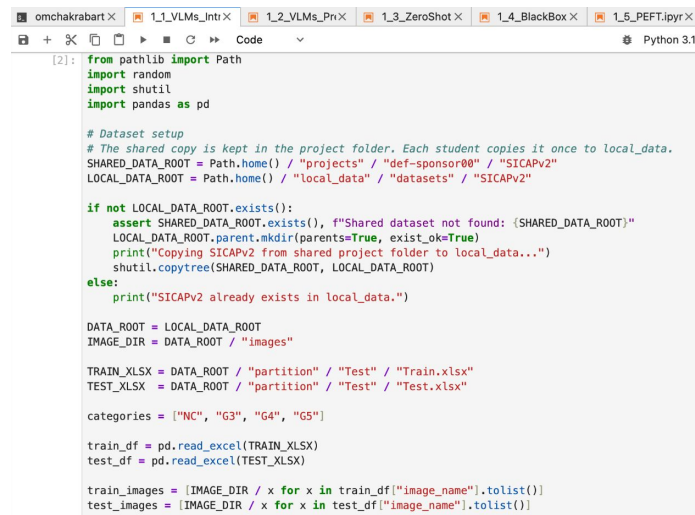
categories = ["NC", "G3", "G4", "G5"]

train_df = pd.read_excel(TRAIN_XLSX)
test_df = pd.read_excel(TEST_XLSX)

train_images = [IMAGE_DIR / x for x in train_df["image_name"].tolist()]
test_images = [IMAGE_DIR / x for x in test_df["image_name"].tolist()]
```

6- Cells and running code

- Notebooks are made of cells : they can be text or python code
- Python code can be executed from the jupyter lab with play button 
- Outputs are added in a new cell after the code executed
- A cell can be executed many times
- Code of a cell can be changed
- Generally, a cell requires that the preceding cells have been executed before



```
[2]: from pathlib import Path
import random
import shutil
import pandas as pd

# Dataset setup
# The shared copy is kept in the project folder. Each student copies it once to local_data.
SHARED_DATA_ROOT = Path.home() / "projects" / "def-sponsor00" / "SICAPv2"
LOCAL_DATA_ROOT = Path.home() / "local_data" / "datasets" / "SICAPv2"

if not LOCAL_DATA_ROOT.exists():
    assert SHARED_DATA_ROOT.exists(), f"Shared dataset not found: {SHARED_DATA_ROOT}"
    LOCAL_DATA_ROOT.parent.mkdir(parents=True, exist_ok=True)
    print("Copying SICAPv2 from shared project folder to local_data...")
    shutil.copytree(SHARED_DATA_ROOT, LOCAL_DATA_ROOT)
else:
    print("SICAPv2 already exists in local_data.")

DATA_ROOT = LOCAL_DATA_ROOT
IMAGE_DIR = DATA_ROOT / "images"

TRAIN_XLSX = DATA_ROOT / "partition" / "Test" / "Train.xlsx"
TEST_XLSX = DATA_ROOT / "partition" / "Test" / "Test.xlsx"

categories = ["NC", "G3", "G4", "G5"]

train_df = pd.read_excel(TRAIN_XLSX)
test_df = pd.read_excel(TEST_XLSX)

train_images = [IMAGE_DIR / x for x in train_df["image_name"].tolist()]
test_images = [IMAGE_DIR / x for x in test_df["image_name"].tolist()]
```

7- Troubleshooting

You can find a troubleshooting section at the very end of the Github's README with common problems you might encounter. Check there first in case need.

1_5_PEFT.ipynb: Parameter-Efficient Fine-Tuning

Topics Covered

- Selective Fine-Tuning
- Parameter-Efficient Adaptation
- Lightweight VLM Adaptation

Learning Outcome

Adapt large pretrained models while updating only a small subset of parameters.

 **Troubleshooting (Click to Expand)**